COMPUTER ENGINEERING / MAJOR PROJECT

CineMind

Movie Recommendation System for Digital Platforms

A data-driven web application for personalized discovery, movie similarity, analytics, and recommendation evaluation.

PREPARED BY	PROJECT DETAILS
Atharva Kamate Third-Year Computer Engineering	Dataset: MovieLens latest-small Submission deadline: 20 October 2026
TOOLS & TECHNOLOGIES	**PROJECT REPOSITORY**
Python | Flask | Pandas | NumPy scikit-learn | HTML/CSS/JavaScript	github.com/atharva07-kamate/ cinemind-movie-recommendation

PROJECT SUBMISSION REPORT | 07 OCTOBER 2026

Explainable recommendations, built from real user ratings.

01 / PROJECT OVERVIEW

Discover relevant movies from real preferences

CineMind is a local-first Flask application that turns MovieLens ratings and movie genres into personalized, explainable recommendations. It combines a web dashboard with recommendation, search, analytics, and evaluation features.

Problem statement

Large digital catalogs make it difficult for users to find content that matches their interests. A recommendation system can use user-item interactions and item attributes to rank useful choices and reduce discovery effort.

Project objectives

+ Analyze user-movie ratings and summarize dataset behavior.
+ Generate personalized recommendations and similar-movie suggestions using explainable machine-learning methods.
+ Combine collaborative, content, and popularity signals into a configurable hybrid ranking.
+ Present results through a responsive web interface and measure quality with offline evaluation metrics.

Dataset profile

MOVIELENS LATEST-SMALL	PROJECT SNAPSHOT
9,742 movies	100,836 ratings from 610 users
5-star rating scale	Genres, tags, and external links
Local CSV files	No external movie API required

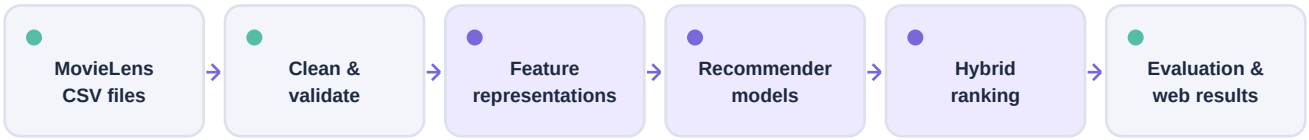
The application loads the real CSV files from the repository's **data/** directory. The source dataset README is included for attribution and license terms. No manually fabricated movie or evaluation data is used.

02 / SYSTEM DESIGN

From CSV files to movie discovery

The Flask application loads and cleans the data once at startup, constructs reusable model representations, and serves HTML pages and JSON endpoints.

Recommendation workflow



A single local dataset powers the web experience and offline quality measurements.

MODULE	WHAT IT DOES
Overview dashboard	Dataset totals, average rating, and popularity-ranked movie cards.
Recommendation workspace	For You, Similar Movies, Popular, and Hybrid modes; supports 5, 10, 15, or 20 results.
Movie discovery	Case-insensitive title search, movie details, rating distribution, and genre-similar titles.
Analytics	Rating distribution, most-rated films, genre coverage, and user rating activity.
Evaluation	Holdout RMSE, MAE, Precision@10, and Recall@10 with a reproducible split.
How it works	Educational pipeline and explanations of preprocessing and model choices.

Application routes include /, /recommendations, /search, /movie/, /analytics, /evaluation, and /about. JSON endpoints support dynamic search, recommendations, movie details, analytics, and evaluation.

03 / RECOMMENDATION METHODOLOGY

Three signals, one explainable ranking

The model uses genre metadata and real rating interactions. Scores are computed from the dataset and recommendations exclude titles already rated by the selected user.

METHOD	IMPLEMENTATION	WHY IT HELPS
Popularity	Bayesian weighted rating blends a movie mean with the global mean. The shrinkage count is based on the 75th percentile of rating counts (at least 10).	Avoids over-ranking movies with only a few ratings.
Content-based	Genre strings are represented with TF-IDF. Cosine similarity compares movie genre vectors; user profiles use movies rated at least 4 stars.	Finds titles with similar genre composition and supports cold-start movie similarity.
Collaborative filtering	A user-item rating matrix is built. Cosine similarity compares user rating patterns; neighboring users' ratings score unseen titles.	Uses the behavior of users with similar preferences.
Hybrid	Normalized scores combine collaborative (0.45), content (0.40), and popularity (0.15). For You uses 0.65 / 0.20 / 0.15.	Balances personal taste, genre match, and rating confidence.

Recommendation explanations

+ Genre similarity explains why two movies are related.
+ Collaborative results identify titles popular with users who share rating patterns.
+ Popularity results use a rating-count-adjusted score rather than an unweighted mean alone.

Cosine similarity compares vector direction, so preference patterns can match even when users have rated different numbers of titles. TF-IDF down-weights common genre tokens relative to more distinguishing ones.

04 / MODEL EVALUATION

Measured on a held-out portion of MovieLens

A deterministic 80/20 split uses seed 42. The metrics below are calculated from the supplied ratings data, not hard-coded example values.

0.9379	0.7367	0.1638	0.1542
RMSE	MAE	PRECISION@10	RECALL@10
Rating error	Absolute error	Relevant in top 10	Relevant items recovered

EVALUATION SETUP	VALUE
Training / test ratings	80,668 / 20,168
Users evaluated for ranking	599 users with at least one held-out relevant item
Relevant rating threshold	4 stars or higher
Top-K cutoff	K = 10
Ranking method	User-based cosine collaborative filtering on training interactions
Rating prediction baseline	Mean of training global, user, and movie averages

Precision@10 is the fraction of the first 10 unseen recommendations that are relevant. Recall@10 is the fraction of a user's held-out relevant movies recovered in the first 10. RMSE penalizes larger rating errors; MAE reports average absolute error. Lower RMSE/MAE indicate closer rating predictions.

Interpretation: these are offline estimates from a random split. A chronological split would better represent predicting future viewing behavior; offline results do not guarantee production performance.

05 / IMPLEMENTATION AND SUBMISSION

A lightweight, CPU-friendly web application

The project keeps model logic separate from Flask routes and uses the included MovieLens files directly. It needs no account database or paid API for its core recommendation features.

LAYER	TOOLS
Backend	Python 3.12, Flask, Gunicorn for Render hosting
Data and ML	Pandas, NumPy, scikit-learn
Frontend	HTML5, CSS3, JavaScript, Jinja templates
Charts and icons	Chart.js and Font Awesome
Hosting and source	Render web service; GitHub repository

Build and verification evidence

CHECK	RESULT
Recommendation model unit checks	6 tests passed: data loading, search, similar movies, user results, invalid IDs, counts, and evaluation.
Flask route smoke check	All seven rendered pages returned HTTP 200 in the local test client.
Recommendation API checks	Search, popular, hybrid, for-you, similar-movie, and invalid-ID behavior verified.
Render deployment	Render dashboard showed the CineMind web service as Deployed in Singapore.

Source repository

github.com/atharva07-kamate/cinemind-movie-recommendation

Repository visibility: private. Share access with a reviewer or make the repository public if they need to browse the source code.

Limitations and future scope

MovieLens latest-small is a 2018 snapshot without demographics. Genre tags are coarse, and new users/items have limited signals. Future work can compare temporal evaluation, item-based or matrix-factorization methods, and optional licensed poster sources.

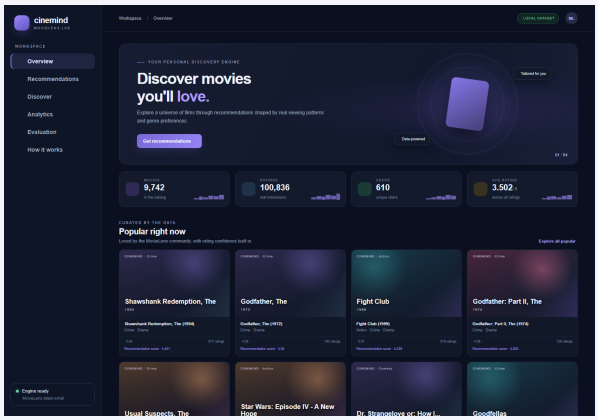
DATASET ACKNOWLEDGMENT

Harper, F. M., and Konstan, J. A. (2015). The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems*, 5(4), Article 19. Dataset source: GroupLens MovieLens. The original dataset README and use terms are preserved in the repository.

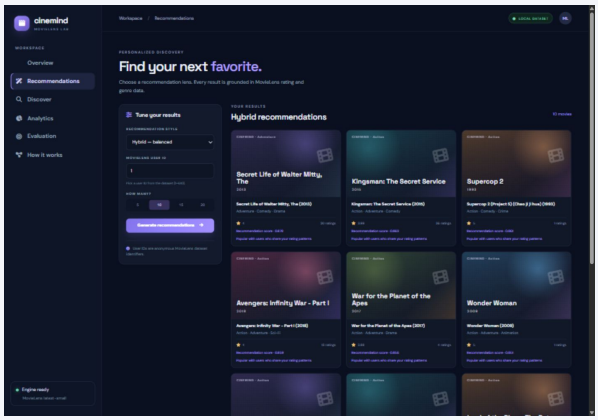
06 / APPLICATION EVIDENCE

Screens from the working CineMind application

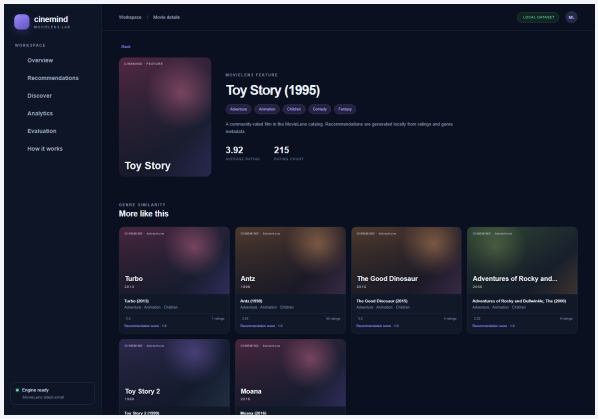
The figures below were captured from the running Flask application using MovieLens latest-small data. The personalized results example uses MovieLens user ID 1.



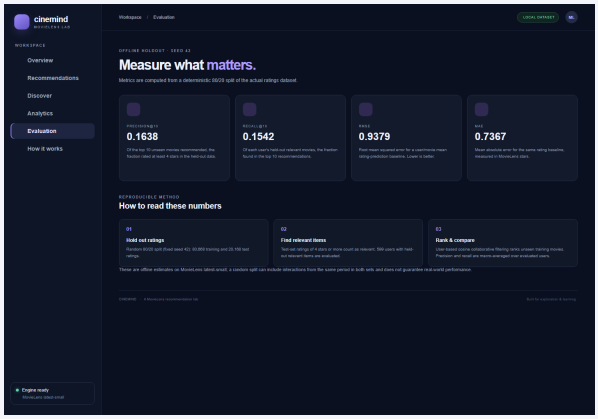
Dashboard — dataset totals and popularity-ranked movie cards.



Hybrid recommendations — generated results for user ID 1 with scores and reasons.



Movie details — Toy Story rating summary and genre-similar suggestions.



Model evaluation — held-out Precision@10, Recall@10, RMSE, and MAE.

Evidence note: the interface uses poster-style placeholders because MovieLens latest-small does not include poster images. Recommendation titles, rating summaries, and evaluation values shown here are derived from the supplied CSV dataset.